

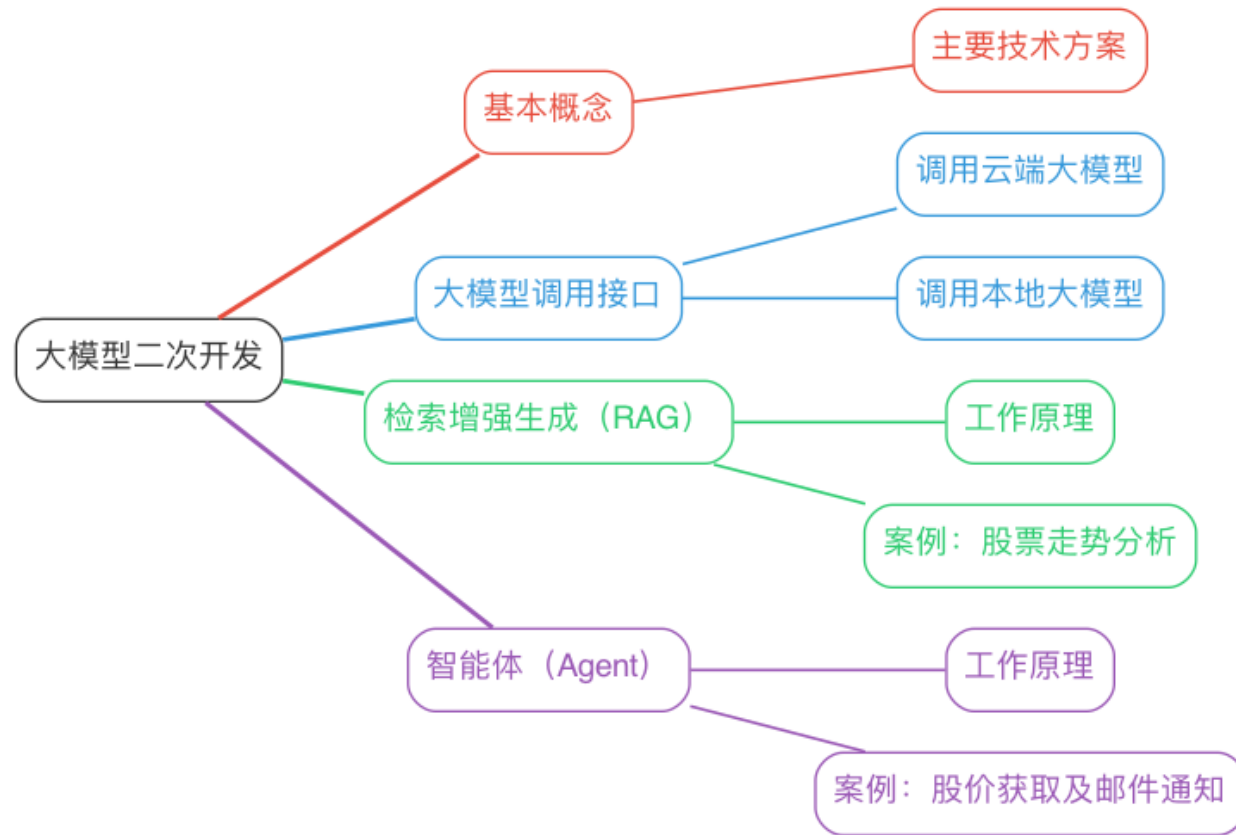


《实用Python编程》

第14讲：大模型二次开发

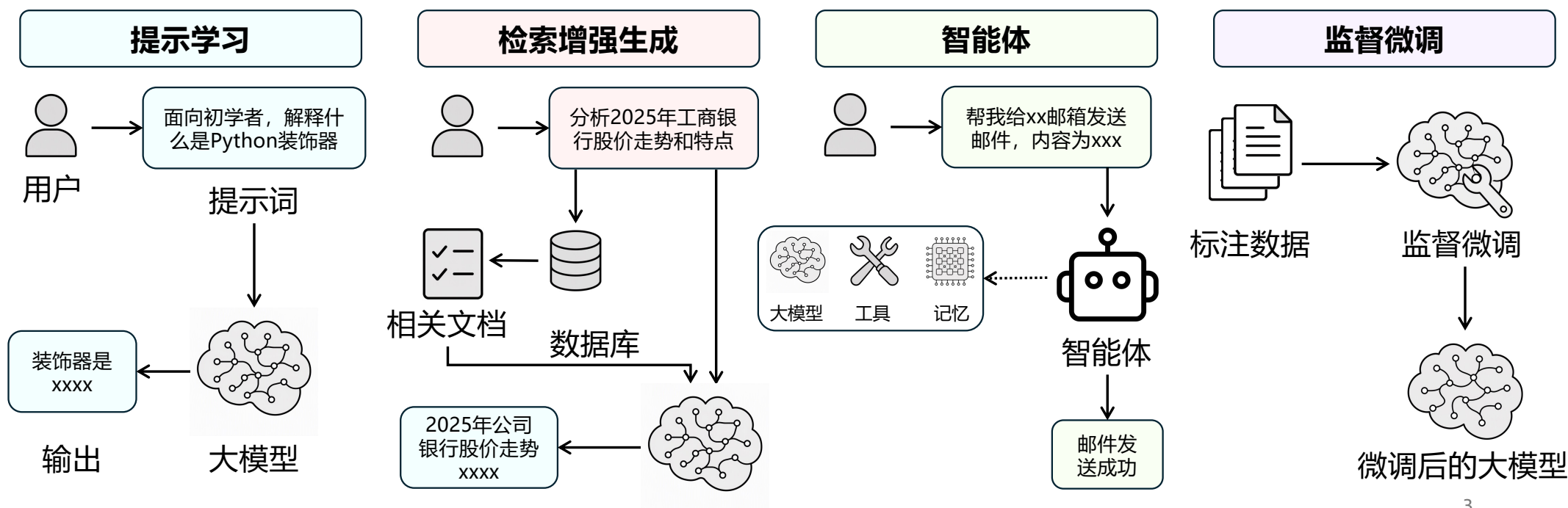
实用Python编程 教研组

提纲



什么是大模型二次开发?

- 针对特定需求或应用场景，对现有大模型进行轻量级定制与能力扩展，使其能够更好地完成指定任务





大模型二次开发的主要技术方案

• 重点讨论检索增强生成与智能体

表 8.3: 大模型二次开发的四种技术方案

技术方案	针对问题	工作原理	数据需求	准确性	技术门槛	适用场景
提示工程	输出不稳定、格式欠规范	上下文学习：通过精心设计提示词，明确角色、任务和输出约束（第7.2.1节）	极低（仅需指令）	中（易有幻觉）	低（写好提示词）	通用任务
检索增强生成	知识时效性不足	开卷考试：先从外部知识库中检索相关片段，再将这些片段随问题一起发送给大模型进行总结生成（第8.3节）	中（需要构建知识库）	高（有据可查）	中（使用向量数据库）	动态知识需求
智能体	无行动执行能力	自主控制：将大模型作为“大脑”负责决策，利用规划、记忆和工具使用，迭代完成复杂任务（第8.4节）	中（需要定义工具接口）	视任务难度而定	偏高（程序设计）	自动化操作
监督微调	垂直领域知识缺乏	知识内化：使用特定格式的“指令-输出”对数据集，训练大模型使其在特定领域、特定语境下有更好表现（第8.5节）	高（需要高质量标注集）	高（在特定任务上）	高（训练大模型）	垂直领域增强



提纲

- 大模型二次开发简介
- 大模型的Python调用接口
- 检索增强生成（RAG）
- 智能体（Agent）

如何调用云端大模型？

- 通过第三方库openai，让用户能以对象调用的方式与云端模型进行远程通信

在Notebook中创建一个OpenAI类对象

```
from openai import OpenAI  
OpenAI(api_key, base_url)
```

"人民大学是哪年建立的？"



中国人民大学成立于1950年。

云端模型



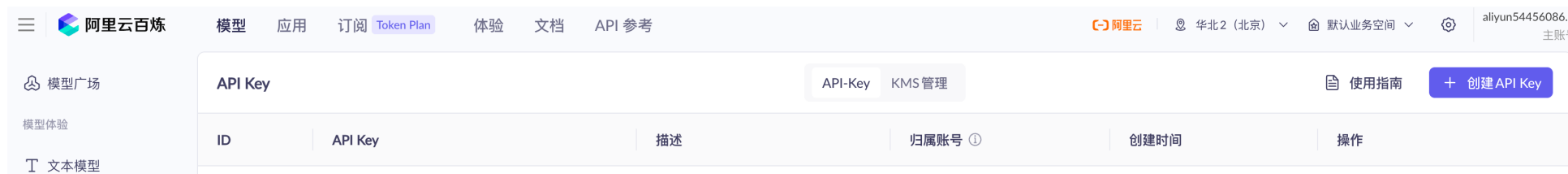
我们以阿里云为例

<https://help.aliyun.com/zh/model-studio/qwen-api-via-openai-chat-completions>



申请你的大模型api_key

- Step 1. 在浏览器中打开阿里云百炼平台，注册账号并（用手机）实名认证 <https://bailian.console.aliyun.com/>
- Step 2. 登录之后，在平台页面依次点击“体验中心” - “密钥管理” - “创建API-KEY”





申请你的大模型api_key

- 点击“+创建API Key”按钮后，会弹出如下所示的对话框

创建API Key

归属业务空间 *

默认业务空间

描述

Project-LLM

11 / 200

权限 *

全部 自定义

取消 确定



保存你的API Key

❗ 离开此对话框后，您将无法取回密钥。
立刻复制或下载您的API密钥。注意妥善保管，任何获取到该密钥的人，都能以您的身份发起服务请求，并产生费用。如果丢失，你可以重置或者创建新的密钥

手动配置 安装并配置 CLI

API Key

复制

API Host

.cn-beijing.maas.aliyuncs.com

OpenAI 兼容地址

https://.cn-beijing.maas.aliyuncs.com/compatible-mode/v1

DashScope

https://.cn-beijing.maas.aliyuncs.com/api/v1

收起

描述	所属业务空间	权限
Project-LLM	默认业务空间	全部

下载 关闭

点击“下载”按钮，相关信息会以csv文件保存到本地：

文件名格式：默认业务空间-apiKey-{project_id}.csv

文件内容：

```
id,xxx  
apiKey,xxx  
apiHost,xxx  
openAiCompatible,xxx  
dashScope,xxx  
description,Project-LLM  
workspaceName,默认业务空间  
workspaceId,xxx
```



从CSV文件中读取目标字段

- 两个目标字段

- apiKey # 调用大模型所需的key
- openAiCompatible # 提供大模型调用服务的网址，即OpenAI的base_url参数

- 利用pandas读取

key_file

id,xxx
apiKey,xxx
apiHost,xxx
openAiCompatible,xxx
dashScope,xxx
description,Project-LLM
workspaceName,默认业务空间
workspaceId,xxx

```
1 import pandas as pd
2
3 project_id = 'Your-Project-ID'
4 key_file = f'默认业务空间-apiKey-{project_id}.csv' # 从
5 # 将CSV文件中的第1列（索引为0的列）设为DataFrame的行索引
6 df = pd.read_csv(key_file, header=None, index_col=0)
7 # 选择列名为 1 的列（即CSV文件中的第2列数据），并转换为字典，格式为 {索引: 值}
8 params = df[1].to_dict()
9 print(params['openAiCompatible'], params['apiKey'])
```



远程调用大模型

• 自定义类LLMClient

```
1 from openai import OpenAI
2
3 class LLMClient():
4     def __init__(self, params):
5         self.client = OpenAI(api_key=params['apiKey'], base_url=params['openAiCompatible'])
6
7     def _process(self, prompt, model="qwen3-max"):
8         messages = [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": prompt}]
9         completion = self.client.chat.completions.create(model=model, messages=messages)
10        resp = completion.to_dict()['choices'][0]['message']['content']
11        return resp
12
13    def answer(self, prompt):
14        return self._process(prompt)
```

调用方式可能随大模型服务更新而改变,
注意参考官方文档
<https://help.aliyun.com/zh/model-studio/qwen-api-reference>

测试代码

```
1 llm = LLMClient(params)
```

```
1 resp = llm.answer("人大是哪年建立的? ")
2 print(resp)
```

中华人民共和国全国人民代表大会（简称“人大”）是中华人民共和国的最高国家权力机关。第一届全国人民代表大会第一次会议于**1954年9月**在北京召开，这标志着全国人民代表大会制度的正式建立。

因此，人大是在**1954年**建立的。

```
1 resp = llm.answer("人民大学是哪年建立的? ")
2 print(resp)
```

中国人民大学创建于1950年。其前身可以追溯到1937年成立的陕北公学，后历经华北联合大学、华北大学等阶段。1950年10月3日，以华北大学为基础，合并了中国政法大学（原朝阳大学）等资源，正式组建成立中国人民大学，是新中国创办的第一所新型正规大学。



本地部署大模型

- 通过huggingface_hub工具包下载模型到本地

```
from huggingface_hub import snapshot_download
```

```
snapshot_download(repo_id="Qwen/Qwen3-0.6B", local_dir="./local_llms/Qwen3-0.6B", endpoint='https://hf-mirror.com')
```

Fetching 10 files: 100%  10/10 [00:00<00:00, 647.63it/s]

- 也可通过镜像网页直接从网页下载
 - <https://hf-mirror.com/Qwen/Qwen3-0.6B/tree/main>

本地模型加载与调用

- 继承`LLMClient`, 重写`__init__`和`_process`方法
- 调用方式和远程保持一致

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2
3 class LocalLLMClient(LLMClient):
4     def __init__(self, model_path):
5         self.tokenizer = AutoTokenizer.from_pretrained(model_path)
6         self.model = AutoModelForCausalLM.from_pretrained(model_path, dtype="auto", device_map="auto")
7
8     def _process(self, prompt):
9         messages = [{"role": "user", "content": prompt}]
10        text = self.tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True, enable_thinking=False)
11        model_inputs = self.tokenizer([text], return_tensors="pt").to(self.model.device)
12        generated_ids = self.model.generate(**model_inputs, max_new_tokens=10000 #, do_sample=False)
13        output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()
14        resp = self.tokenizer.decode(output_ids, skip_special_tokens=True)
15        return resp
16
17 # 测试代码
18 model_path = 'local_llms/Qwen3-0.6B'
19 llm = LocalLLMClient(model_path)
20 resp = llm.answer("最小的鸟类是什么? ")
21 print(resp)
```

最小的鸟类是**鸚鵡**。



大模型的两种调用模式对比

- 各有利弊

调用模式	成本	能力	隐私	门槛
云端	按token计费	较强	数据上传云端	联网即可
本地	硬件成本与运行成本	受硬件限制， 相对偏弱	数据在本地	硬件与编码能力 要求



检索增强生成 (RAG)

- 大模型天然短板
 - 未见过最新及私有数据
- Retrieval Augmented Generation (RAG)
 - 给大模型外挂知识库
 - 好比“开卷考试”：先查资料，再作答

RAG基本流程

1、文档库构建

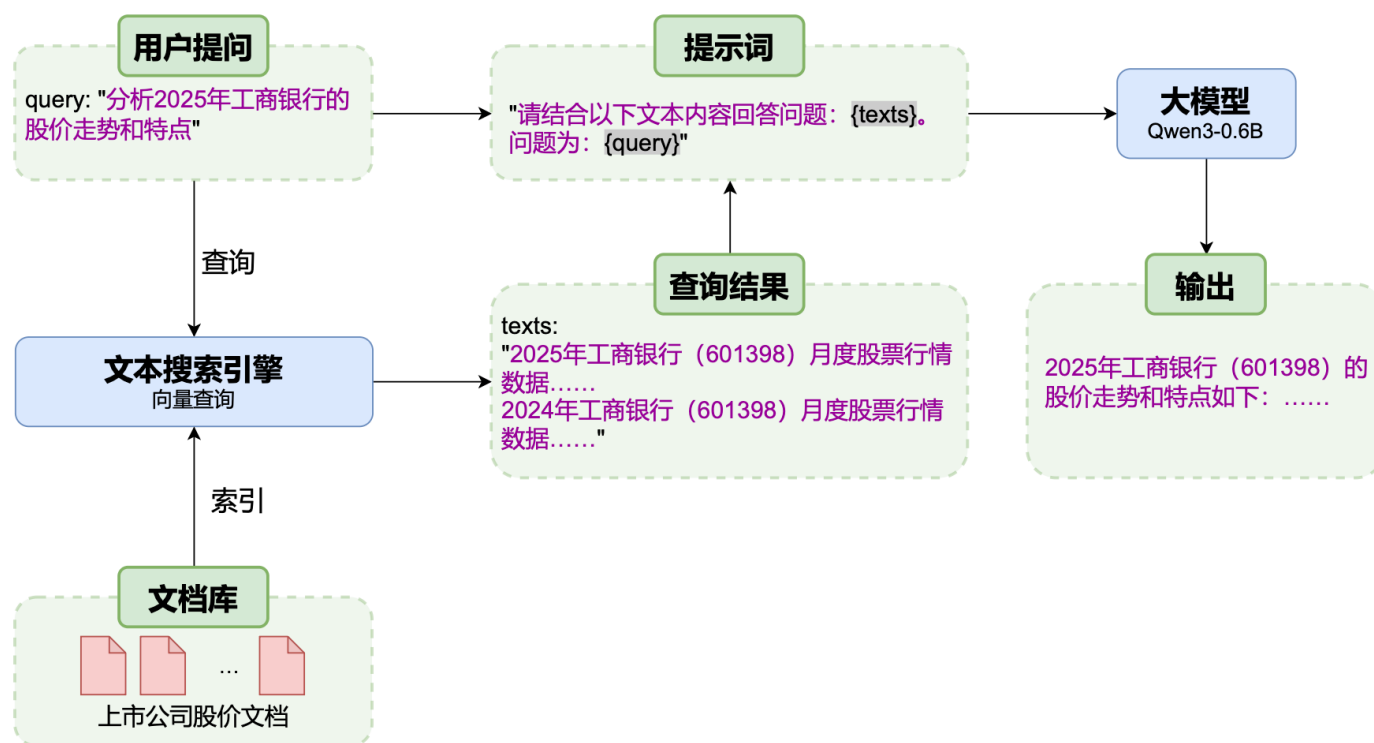
2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答

案例：用Qwen3-0.6B分析股价走势



RAG基本流程

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答



第1步. 构建股票数据库

• 基于腾讯财经接口, 定义TFShare类

```
1 import pandas as pd
2 import requests
3
4 RENAME_PERIOD = {'daily': 'day', 'weekly': 'week', 'monthly': 'month'}
5
6 class TFShare():
7     @staticmethod
8     def __get_market_code(stock_id):
9         """由6位股票代码推断股票所在交易所: sh (上交所) /sz (深交所) /bj (北交所)"""
10        if stock_id.startswith(('60', '68', '90', '11', '13')):
11            return 'sh'
12        if stock_id.startswith(('00', '30', '20', '12')):
13            return 'sz'
14        return 'bj'
15
16        """腾讯财经行情接口, stock_zh_a_hist方法与akshare的同名函数用法一致。"""
17    @staticmethod
18    def stock_zh_a_hist(symbol, period='daily', start_date='19700101', end_date='20500101', adjust='qfq'):
19        assert(adjust in ['', 'qfq', 'hfq'])
20        code = TFShare.__get_market_code(symbol) + symbol
21        period = RENAME_PERIOD[period]
22        begin, finish = [f'{d[:4]}-{d[4:6]}-{d[6:8]} ' for d in (start_date, end_date)] # 腾讯接口的日期格式
23        url = 'https://web.ifzq.gtimg.cn/appstock/app/fqkline/get'
24        param = f'{code},{period},{begin},{finish},1000,{adjust}'
25        klines = []
26        try:
27            response = requests.get(url, params={'param': param}, timeout=10)
28            response.raise_for_status() # 如果返回状态码不是200, 则抛出异常
29            result = response.json()
30            klines = result['data'][code].get(f'{adjust}-{period}', [])
31        except Exception as e:
32            print(f"请求失败, 错误信息: {e}")
33        columns = ['日期', '开盘', '收盘', '最高', '最低', '成交量']
34        df = pd.DataFrame([kline[:6] for kline in klines], columns=columns)
35        df['日期'] = pd.to_datetime(df['日期'], errors='coerce').dt.date
36        for col in columns[1:]:
37            df[col] = pd.to_numeric(df[col], errors='coerce')
38        return df
```

RAG基本流程

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答



第1步. 构建股票数据库

• 对TFShare类进行测试

```
year = 2025
stock_id = '601398'
df = TFShare.stock_zh_a_hist(symbol=stock_id, period='monthly', start_date=f'{year}0101', end_date=f'{year}1231')
```

	日期	开盘	收盘	最高	最低	成交量
0	2025-01-27	6.292	6.345	6.375	5.925	64319547.0
1	2025-02-28	6.345	6.395	6.665	6.135	63131259.0
2	2025-03-31	6.395	6.415	6.465	6.225	67024784.0
3	2025-04-30	6.395	6.535	6.865	6.125	84388556.0
4	2025-05-30	6.535	6.595	6.825	6.455	53944359.0
5	2025-06-30	6.575	7.115	7.285	6.545	59525517.0
6	2025-07-31	7.115	7.250	7.885	7.090	83537190.0
7	2025-08-29	7.250	7.120	7.590	7.010	81947790.0
8	2025-09-30	7.120	6.990	7.290	6.810	92550903.0
9	2025-10-31	6.940	7.470	7.710	6.890	66212344.0
10	2025-11-28	7.480	7.800	8.090	7.470	60918073.0
11	2025-12-31	7.740	7.761	7.840	7.530	54292463.0

RAG基本流程

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答



第1步. 构建股票数据库

- 在线获取股票数据并保存为一个本地文件

```
1 years = [2024, 2025] # 要获取数据的年份
2 stock_list = [('601398', '工商银行'), ('600519', '贵州茅台'), ('300750', '宁德时代')] # 要获取数据的(股票代码、公司名称) 二元组列表
3 stock_data_path = 'stock_data_tfshare.txt' # 存储地址
4 with open(stock_data_path, 'w') as fw:
5     for stock_id, company in stock_list:
6         for year in years:
7             df = TFSHARE.stock_zh_a_hist(symbol=stock_id, period='monthly',
8                 start_date=f'{year}0101', end_date=f'{year}1231', adjust='qfq')
9             texts = [f"日期{row['日期']}, 收盘价{row['收盘']}元。" for _, row in df.iterrows()]
10            texts.insert(0, f'{year}年{company} ({stock_id}) 月度股票行情数据。')
11            fw.write(''.join(texts) + '\n')
```



```
1 2024年工商银行 (601398) 月度股票行情数据。日期2024-01-31, 收盘价4.41元。... 12-31, 收盘价6.47元。
2 2025年工商银行 (601398) 月度股票行情数据。日期2025-01-27, 收盘价6.51元。... 12-31, 收盘价7.93元。
3 2024年贵州茅台 (600519) 月度股票行情数据。日期2024-01-31, 收盘价1498.59元。... 12-31, 收盘价1472.44元。
4 2025年贵州茅台 (600519) 月度股票行情数据。日期2025-01-27, 收盘价1383.43元。... 12-31, 收盘价1377.18元。
5 2024年宁德时代 (300750) 月度股票行情数据。日期2024-01-31, 收盘价139.8元。... 12-31, 收盘价259.24元。
6 2025年宁德时代 (300750) 月度股票行情数据。日期2025-01-27, 收盘价251.46元。... 12-31, 收盘价367.26元。
```

每行是一篇“文档”：一只股票的年度月度行情，共 6 篇

RAG基本流程

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答



第2步. 向量化索引

RAG基本流程

- 实现一个基于向量查询的文本搜索引擎类 `TextSearchEngine`

```
1 import numpy as np
2 from sentence_transformers import SentenceTransformer
3
4 class TextSearchEngine():
5     def __init__(self, text_encoder_path="./fm/all-MiniLM-L6-v2"):
6         self.encoder = SentenceTransformer(text_encoder_path)
7         self.embeddings = np.array([])
8         self.texts = []
9
10    def _txt2vec(self, texts):
11        return self.encoder.encode(texts)
12
13    def index(self, texts): # 新增文本向量表示
14        keys = [text.split('.')[0] for text in texts] # 将每条记录的第一句话向量化
15        new_embeddings = self._txt2vec(keys)
16        if len(self.embeddings) == 0:
17            self.embeddings = new_embeddings
18        else:
19            self.embeddings = np.vstack((self.embeddings, new_embeddings))
20        self.texts.extend(texts)
21
22    def search(self, query, top_k=2): # 查找相似文本
23        query_embedding = self._txt2vec([query])
24        # 此处模型已经保证embedding维数为1
25        scores = np.dot(self.embeddings, query_embedding.T).flatten()
26        top_indices = np.argsort(scores)[-top_k:][::-1]
27        hits = [self.texts[i] for i in top_indices]
28        return hits
```

基于MiniLM(5.4节)
对文本进行向量化

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答



第2步. 向量化索引

- 将股价数据库向量化

```
1 stock_data_path = 'stock_data_tfshare.txt'
2 texts = open(stock_data_path).readlines()
3 texts = [line.strip('\r\n') for line in texts]
4 searcher = TextSearchEngine()
5 searcher.index(texts)
```

RAG基本流程

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答



第3步. 相关文档检索

- 找出与当前查询(query)相关的股票文档

```
1 query = "分析2025年工商银行的股价走势和特点"
2 hits = searcher.search(query=query, top_k=2)
3 for text in hits:
4     print(text)
```

2025年工商银行（601398）月度股票行情数据。日期2025-01-27，收盘价6.345元。日期2025-02-28，收盘价6.395元。日期2025-03-31，收盘价6.415元。日期2025-04-30，收盘价6.535元。日期2025-05-30，收盘价6.595元。日期2025-06-30，收盘价7.115元。日期2025-07-31，收盘价7.25元。日期2025-08-29，收盘价7.12元。日期2025-09-30，收盘价6.99元。日期2025-10-31，收盘价7.47元。日期2025-11-28，收盘价7.8元。日期2025-12-31，收盘价7.761元。

2024年工商银行（601398）月度股票行情数据。日期2024-01-31，收盘价4.245元。日期2024-02-29，收盘价4.405元。日期2024-03-29，收盘价4.355元。日期2024-04-30，收盘价4.505元。日期2024-05-31，收盘价4.505元。日期2024-06-28，收盘价4.775元。日期2024-07-31，收盘价5.232元。日期2024-08-30，收盘价5.362元。日期2024-09-30，收盘价5.562元。日期2024-10-31，收盘价5.422元。日期2024-11-29，收盘价5.532元。日期2024-12-31，收盘价6.302元。

RAG基本流程

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答



第4步. 提示词拼接

- 将检索结果hit与问题query一起放到提示词中

```
context = '\n'.join(hits)
prompt = f"请结合以下文本内容回答问题: {context}。 \n问题为: {query}"
```

RAG基本流程

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答



第5步. 模型生成回答

RAG基本流程

- 创建一个LocalLLMClient对象
 - `llm = LocalLLMClient('local_llms/Qwen3-0.6B')`
- 调用该对象回答问题
 - `resp = llm.answer(prompt)`

1、文档库构建

2、向量化索引

3、相关文档检索

4、提示词拼接

5、模型生成回答

RAG前：模型没有2025年数据，只能泛泛而谈

2025年，中国工商银行（ICBC）的股价走势和特点可以从以下几个方面进行分析：

一、市场背景与行业环境

1. **宏观经济形势**
 - 2025年是中国经济进入高质量发展阶段的关键年，政府持续推动“双循环”战略，加强金融基础设施建设，预计央行将继续加大货币政策工具的使用，推动信贷增长。
 - 工商银行作为国有大型商业银行，其业务规模和影响力将进一步扩大，尤其是在普惠金融、绿色金融、数字金融等领域。
2. **政策支持**
 - 国家推动“十四五”规划和2035年远景目标，强调金融体系的现代化和普惠性。工商银行在数字化转型、金融科技（FinTech）方面的投入将加速，提升服务实体经济的能力。
3. **行业竞争格局**
 - 工商银行作为国内领先的国有银行，其市场份额和客户基础稳固，但竞争压力可能持续。同时，其他国有银行（如建设银行、招商银行）也在加速布局，形成差异化竞争。

二、2025年股价走势预测

1. **短期（2025Q1-Q2）**
 - **市场情绪**：市场对政策支持和行业前景的乐观情绪可能持续，尤其是对工商银行在数字化转型和绿色金融方面的表现。
 - **股价表现**：若工商银行在2025年Q1-Q2期间继续稳定增长，股价可能保持在中上水平，但受政策影响，可能有波动。
 - **关键因素**：政策红利、行业增长预期、客户留存率提升等。

RAG后：基于真实数据给出准确的分析

2025年工商银行（601398）的股价走势和特点如下：

1. **股价波动**：2025年1月27日收盘价为6.51元，2月28日为6.56元，3月31日为6.58元，4月30日为6.7元，5月30日为6.76元，6月30日为7.28元，7月31日为7.42元，8月29日为7.29元，9月30日为7.16元，10月31日为7.64元，11月28日为7.97元，12月31日为7.93元。可以看出，股价在2025年呈现了一定的波动，尤其是在6月和7月，价格有所上涨，而12月则略有回落。
2. **趋势分析**：从数据来看，2025年12月的收盘价为7.93元，比11月的7.97元略低，但整体趋势较为平稳，没有明显的大幅波动。此外，股价在6月和7月上涨，显示出一定的上升趋势，而12月则有所回落，可能反映了市场环境的变化或投资者情绪的波动。
3. **特点总结**：2025年工商银行的股价走势呈现出波动性较强，尤其是在6月和7月，价格有所上涨，而12月则有所回落。整体来看，股价在2025年保持了一定的稳定性和韧性，但存在一定的波动性。



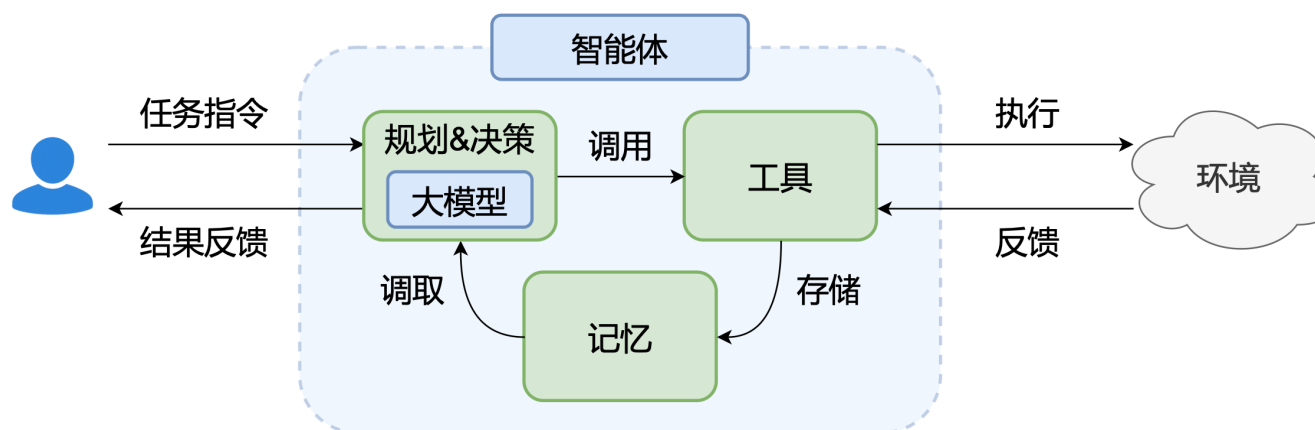
课堂练习-8-1

TextSearchEngine类中的index方法不检查文档是否已经在库中，从而有文档重复入库的风险。请修改该方法的实现，只对尚未入库的文本做向量化，已在库中的则跳过。

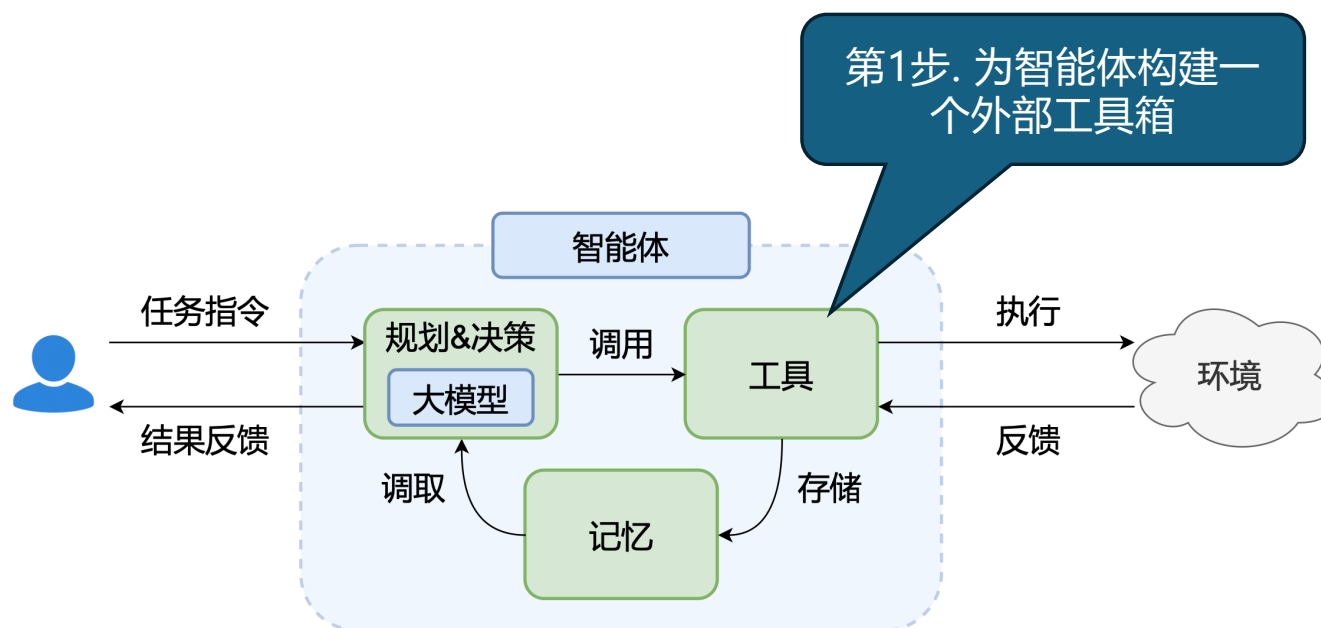
```
1 import numpy as np
2 from sentence_transformers import SentenceTransformer
3
4 class TextSearchEngine():
5     def __init__(self, text_encoder_path="./fm/all-MiniLM-L6-v2"):
6         self.encoder = SentenceTransformer(text_encoder_path)
7         self.embeddings = np.array([])
8         self.texts = []
9
10    def _txt2vec(self, texts):
11        return self.encoder.encode(texts)
12
13    def index(self, texts): # 新增文本向量表示
14        keys = [text.split('.')[0] for text in texts] # 将每条记录的第一句话向量化
15        new_embeddings = self._txt2vec(keys)
16        if len(self.embeddings) == 0:
17            self.embeddings = new_embeddings
18        else:
19            self.embeddings = np.vstack((self.embeddings, new_embeddings))
20        self.texts.extend(texts)
21
22    def search(self, query, top_k=2): # 查找相似文本
23        query_embedding = self._txt2vec([query])
24        # 此处模型已经保证embedding范数为1
25        scores = np.dot(self.embeddings, query_embedding.T).flatten()
26        top_indices = np.argsort(scores)[-top_k:]
27        hits = [self.texts[i] for i in top_indices]
28        return hits
```

智能体 (Agent)

- 本质：让大模型学会使用工具，使其从一个对话系统变成AI助理
- 智能体是以大模型为内核的系统架构
 - 并不是一个（新的）大模型
 - ReAct (**R**easoning + **A**cting)



案例：自动化股价获取及邮件通知



第1步. 构建工具箱

- 先搭建一个工具箱架构，再往里加工具 (skill)

```
1 class AgentToolkit():
2     def __init__(self):
3         self.tools = {}
4
5     def add(self, tool_func, tool_desp):
6         self.tools[tool_func.__name__] = {'func': tool_func, 'info': tool_desp}
7
8     def get(self, tool_name):
9         return self.tools.get(tool_name, {}).get('func', None)
10
11    def describe(self):
12        desp = [f"{tool_name}: {tool['info']}" for tool_name, tool in self.tools.items()]
13        return "\n".join(desp)
14
15    def hello(name):
16        return f'Hello {name}'
17
18    atk = AgentToolkit()
19    atk.add(max, 'compare two input numbers and returns the larger')
20    atk.add(hello, 'returns a hello message to a given name')
21    print(atk.describe())
```

测试代码

max: compare two input numbers and returns the larger
hello: returns a hello message to a given name



第1步. 构建工具箱

- 通过自定义函数实现一个能读取股价的skill
 - 再次使用“智能体案例”中的股票查询接口

```
1 from datetime import date, timedelta
2 #import akshare as ak
3
4 def get_yesterday_stock(stock_id):
5     target_date = date.today() - timedelta(days=1) # 以调用时刻反推昨日
6     target_date = target_date.strftime("%Y%m%d")
7     #df = ak.stock_zh_a_hist(symbol=stock_id, period='daily', start_date=target_date, end_date=target_date, adjust='qfq')
8     df = TFSHare.stock_zh_a_hist(symbol=stock_id, period='daily', start_date=target_date, end_date=target_date, adjust='qfq')
9     record = df.loc[0].to_dict()
10    record['日期'] = record['日期'].strftime("%Y%m%d")
11    record = [f"{key}{value}" for key, value in record.items()]
12    record = ', '.join(record) + '。'
13    return record
14
15 # 函数调用示例
16 print(get_yesterday_stock('601398'))
```

日期20260902, 开盘8.19, 收盘8.22, 最高8.29, 最低8.14, 成交量3717999.0。

第1步. 构建工具箱

- 再实现一个发送邮件的skill
 - 注意用真实值替代代码中的示意性字符

```
1 import smtplib
2 from email.mime.text import MIMEText
3
4 def send_email(subject: str, content: str, receiver: str):
5     sender = "xxxx@xxx.com" # 发件箱
6     authcode = "xxxx" # 发件箱授权码 (注意不是登陆密码), 在邮箱SMTP设置页面查看
7     smtp_host = "xxxx" # 发件箱SMTP服务器地址 (新浪为smtp.sina.com, 163为smtp.163.com, qq为smtp.qq.com)
8     msg = MIMEText(content, "plain", "utf-8") # 创建邮件对象
9     msg["Subject"], msg["From"], msg["To"] = subject, sender, receiver
10    try:
11        with smtplib.SMTP_SSL(smtp_host, 465) as server:
12            server.login(sender, authcode)
13            server.send_message(msg)
14        return "邮件已发送"
15    except Exception as e:
16        return f"邮件发送失败: {e}"
17
18 # 函数调用示例
19 send_email(subject="test", content="hello world", receiver="xxx@126.com")
```

设置3秒后超时

`smtplib.SMTP_SSL(smtp_host, 465, timeout=3)`



第1步. 构建工具箱

- 将两个skill加到工具箱中，完成工具箱配置

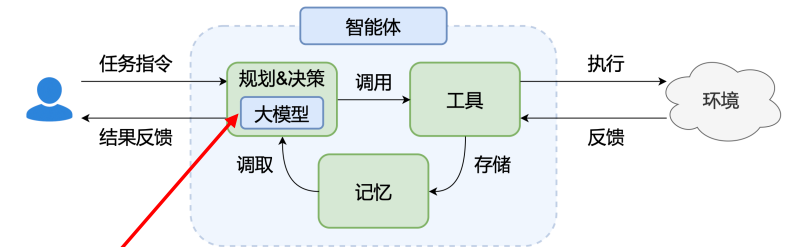
```
1 toolkit = AgentToolkit()
2 toolkit.add(get_yesterday_stock, "获取指定股票的昨日交易数据，参数stock_id表示股票代码")
3 toolkit.add(send_email, "发送电子邮件，输入邮件主题和内容，发送邮件到指定收件人。参数subject表示邮件主题，content表示邮件内容，receiver表示收件人邮箱地址")
4 print(toolkit.describe())
```

get_yesterday_stock: 获取指定股票的昨日交易数据，参数stock_id表示股票代码

send_email: 发送电子邮件，输入邮件主题和内容，发送邮件到指定收件人。参数subject表示邮件主题，content表示邮件内容，receiver表示收件人邮箱地址

第2步. 搭建智能体

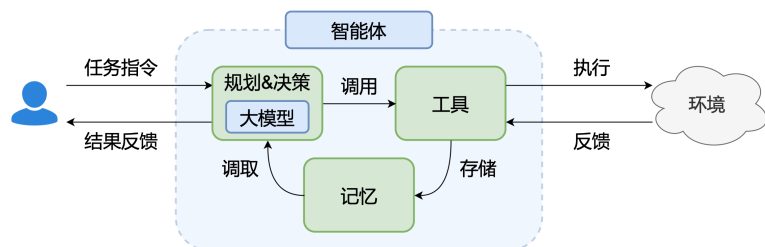
- 通过提示工程 (Prompt Engineering) 让大模型承担智能体“大脑”的角色



```
1 def apply_chat_template(prompt, toolkit_desp, previous_state=''):
2     prompt_template = f"""
3     你是一个智能助理，能够调用外部工具来完成相关的任务。
4     可以使用的工具如下：
5     {toolkit_desp}
6
7     你的回答必须是以下两种形式之一，且不能同时进行：
8     1、如果你需要调用工具来获取信息，按照“行动：[工具名称]；[JSON格式，参数名:参数值]”的格式进行回答。
9     2、如果你已经有足够的信息来回答用户的问题，按照“回答：[你的回答内容]”的格式进行回答。
10
11     任务如下：
12     {prompt}
13     之前返回结果：
14     {previous_state}
15     """
16     return prompt_template
17
18 task_prompt = "查询工商银行（股票代码：601398）昨日的股票交易数据，并将数据发送到邮箱xxxx@ruc.edu.cn"
19 res = apply_chat_template(task_prompt, toolkit.describe())
20 print(res)
```

第2步. 搭建智能体

- 实现智能体的主体架构
 - 在Agent初始化时指定其可调用的大模型 (llm) 和工具箱 (toolkit)



```

1 import json
2 from openai import OpenAI
3
4 class Agent:
5     def __init__(self, llm, toolkit):
6         self.toolkit = toolkit
7         self.llm = llm
8         self.max_steps = 5
9
10    def _process(self, prompt):
11        previous_results = []
12        tk_desp = self.toolkit.describe()
13        for i in range(self.max_steps):
14            full_prompt = apply_chat_template(prompt, tk_desp, '\n'.join(previous_results))
15            resp = self.llm.answer(full_prompt).strip()
16            print(f"Step {i+1}. llm> {resp}")
17            if resp.startswith("回答: "):
18                return {'status': True, 'message': resp[len("回答: "):]}
19            elif resp.startswith("行动: "):
20                action_part = resp[len("行动: "):]
21                tool_name, param_str = action_part.split('; ', 1)
22                param_str = json.loads(param_str)
23                tool_func = self.toolkit.get(tool_name)
24                if tool_func:
25                    res = tool_func(**param_str)
26                    previous_results.append(res) # 收集外部工具的执行结果
27                else:
28                    return {'status': False, 'message': f"未找到工具 {tool_name}"}
29            else:
30                return {'status': False, 'message': "无法解析的响应格式"}
31        return {'status': False, 'message': "达到最大步骤数, 未能得到最终回答。"}
32
33    def answer(self, prompt):
34        resp = self._process(prompt)
35        return resp['message']

```

用列表存储记忆

调用工具



第3步. 使用智能体

• 创建一个Agent对象，并调用

```
1 task_prompt = "查询工商银行（股票代码：601398）昨日的股票交易数据，并将数据发送到邮箱xirong@ruc.edu.cn"
2 agent = Agent(llm, toolkit)
3 resp = agent.answer(task_prompt)
4 resp
```

Step 1. llm> 行动: get_yesterday_stock;{"stock_id": "601398"}

Step 2. llm> 行动: send_email;{"subject": "工商银行（601398）昨日股票交易数据", "content": "日期: 20260904\n开盘价: 8.07\n收盘价: 8.13\n最高价: 8.21\n最低价: 8.06\n成交量: 2438905.0", "receiver": "xirong@ruc.edu.cn"}

Step 3. llm> 回答: 邮件已发送。

'邮件已发送。'

代码执行后，在
指定邮箱中收到
了股票信息

工商银行（601398）昨日股票交易数据

 **Li, Xirong (RUC)** xirong@ruc.edu.cn
发至 Li, Xirong (RUC) 收起

发件人: Li, Xirong (RUC)<xirong@ruc.edu.cn>
收件人: Li, Xirong (RUC)<xirong@ruc.edu.cn>
主题: 工商银行（601398）昨日股票交易数据
时间: 2026年09月05日（周六） 21:39

日期: 20260904
开盘价: 8.07
收盘价: 8.13
最高价: 8.21
最低价: 8.06
成交量: 2438905.0



第3步． 使用智能体

- 智能体的自主性与灵活性

对于不需要工具即可完成任务，可以像普通大模型一样进行回答

```
1 resp = agent.answer('人民大学的前身是什么')  
2 print(resp)
```

Step 1. llm> 回答：中国人民大学的前身是1937年创办的陕北公学，后历经华北联合大学、华北大学等阶段，于1950年正式定名为中国人民大学。
中国人民大学的前身是1937年创办的陕北公学，后历经华北联合大学、华北大学等阶段，于1950年正式定名为中国人民大学。

对于超出其能力范围的任务，也会主动拒绝并告知用户原因

```
1 task_prompt = "我明天从北京去深圳出差，请订一张早上10点出发的直飞机票。"  
2 resp = agent.answer(task_prompt)  
3 print(resp)
```

Step 1. llm> 回答：我无法为您预订机票，因为当前没有提供订票工具。建议您通过航空公司官网或旅行平台自行预订。
我无法为您预订机票，因为当前没有提供订票工具。建议您通过航空公司官网或旅行平台自行预订。

随堂练习-8-2

- 在当前代码中，工具函数的执行（第25行）没有任何保护。一旦工具内部抛出异常，整个智能体会立即终止，用户既得不到回答，也看不到已经完成的中间步骤。请为智能体增加执行阶段的容错能力。

```
1 import json
2 from openai import OpenAI
3
4 class Agent:
5     def __init__(self, llm, toolkit):
6         self.toolkit = toolkit
7         self.llm = llm
8         self.max_steps = 5
9
10    def _process(self, prompt):
11        previous_results = []
12        tk_desp = self.toolkit.describe()
13        for i in range(self.max_steps):
14            full_prompt = apply_chat_template(prompt, tk_desp, '\n'.join(previous_results))
15            resp = self.llm.answer(full_prompt).strip()
16            print(f"Step {i+1}. llm> {resp}")
17            if resp.startswith("回答: "):
18                return {'status': True, 'message': resp[len("回答: "):]}
19            elif resp.startswith("行动: "):
20                action_part = resp[len("行动: "):]
21                tool_name, param_str = action_part.split('; ', 1)
22                param_str = json.loads(param_str)
23                tool_func = self.toolkit.get(tool_name)
24                if tool_func:
25                    res = tool_func(**param_str)
26                    previous_results.append(res) # 收集外部工具的执行结果
27                else:
28                    return {'status': False, 'message': f"未找到工具 {tool_name}"}
29            else:
30                return {'status': False, 'message': "无法解析的响应格式"}
31        return {'status': False, 'message': "达到最大步骤数，未能得到最终回答。"}
32
33    def answer(self, prompt):
34        resp = self._process(prompt)
35        return resp['message']
```



随堂练习-8-3

案例中的智能体无法订票。现定义一个book_flight函数，描述为预订飞机票工具，但实际仅返回“订票已完成”。将该工具加入智能体工具箱之后，智能体会如何响应？这显示出什么风险？

```
def book_flight(departure_city, arrival_city, departure_time, flight_type):  
    return "订票已完成"
```

```
toolkit.add(book_flight, "预订飞机票。参数departure_city表示出发地，arrival_city表示目的地，departure_time表示出发时间，  
flight_type表示航班类型")
```